

Rodriguez Joaquin

Introducción Teórica

En el desarrollo de aplicaciones con MongoDB, existen dos enfoques principales para gestionar la comunicación:

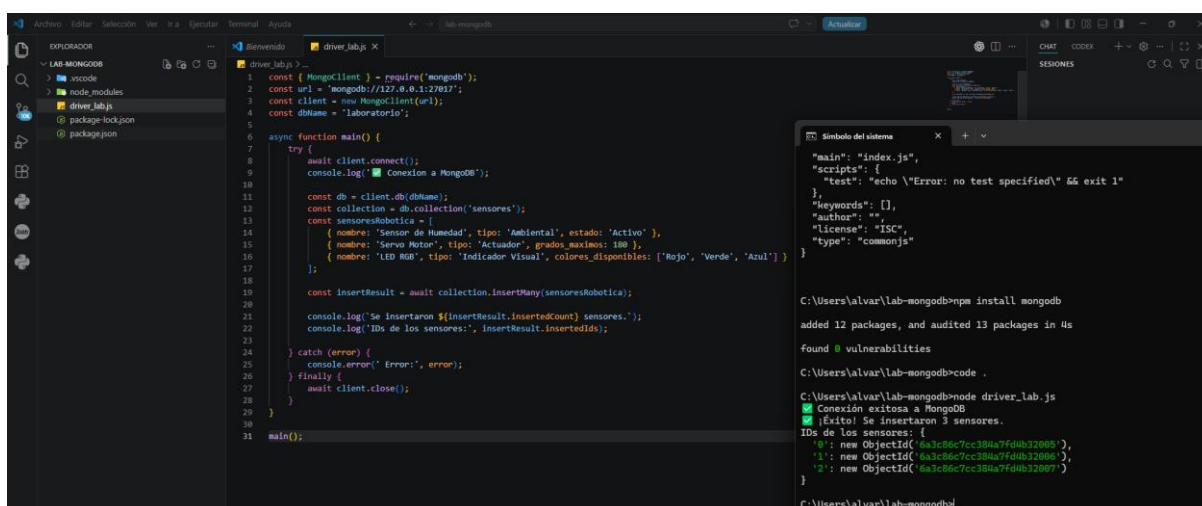
1. **Driver Nativo:** Provee la comunicación más directa con el motor de base de datos. Se destaca por ofrecer el máximo control sobre las consultas y una mayor eficiencia al no tener capas intermedias.
2. **Mongoose (ODM):** Actúa como una capa de abstracción sobre el driver. Permite definir esquemas para organizar los datos y forzar validaciones directamente en el código de la aplicación.

Parte 1: Implementación con Driver Nativo

El driver oficial (**mongodb**) es ideal para aplicaciones sencillas o procesos de alto volumen donde el rendimiento es crítico.

Instrucciones:

1. Inicialicen el proyecto e instalen el paquete: **npm install mongodb**.
2. Creen un archivo **driver_lab.js** para conectarse a la base de datos **laboratorio**.
3. Usando el método **insertMany**, carguen un array con tres sensores de robótica (ej: Sensor de Humedad, Servo Motor, LED RGB).



```
1 const { MongoClient } = require('mongodb');
2 const url = 'mongodb://127.0.0.1:27017';
3 const client = new MongoClient(url);
4 const dbName = 'laboratorio';
5
6 async function main() {
7   try {
8     await client.connect();
9     console.log('Conexión a MongoDB');
10
11     const db = client.db(dbName);
12     const collection = db.collection('sensores');
13     const sensoresRobotica = [
14       { nombre: 'Sensor de Humedad', tipo: 'Ambiental', estado: 'Activo' },
15       { nombre: 'Servo Motor', tipo: 'Actuador', grados_maximos: 180 },
16       { nombre: 'LED RGB', tipo: 'Indicador Visual', colores_disponibles: ['Rojo', 'Verde', 'Azul'] }
17     ];
18
19     const insertResult = await collection.insertMany(sensoresRobotica);
20     console.log(`Se insertaron ${insertResult.insertedCount} sensores.`);
21     console.log(`IDs de los sensores:`, insertResult.insertedIds);
22
23     catch (error) {
24       console.error('Error:', error);
25     } finally {
26       await client.close();
27     }
28   }
29 }
30
31 main();
```

```
C:\Users\alvar\lab-mongodb>npm install mongodb
added 12 packages, and audited 13 packages in 4s
found 0 vulnerabilities

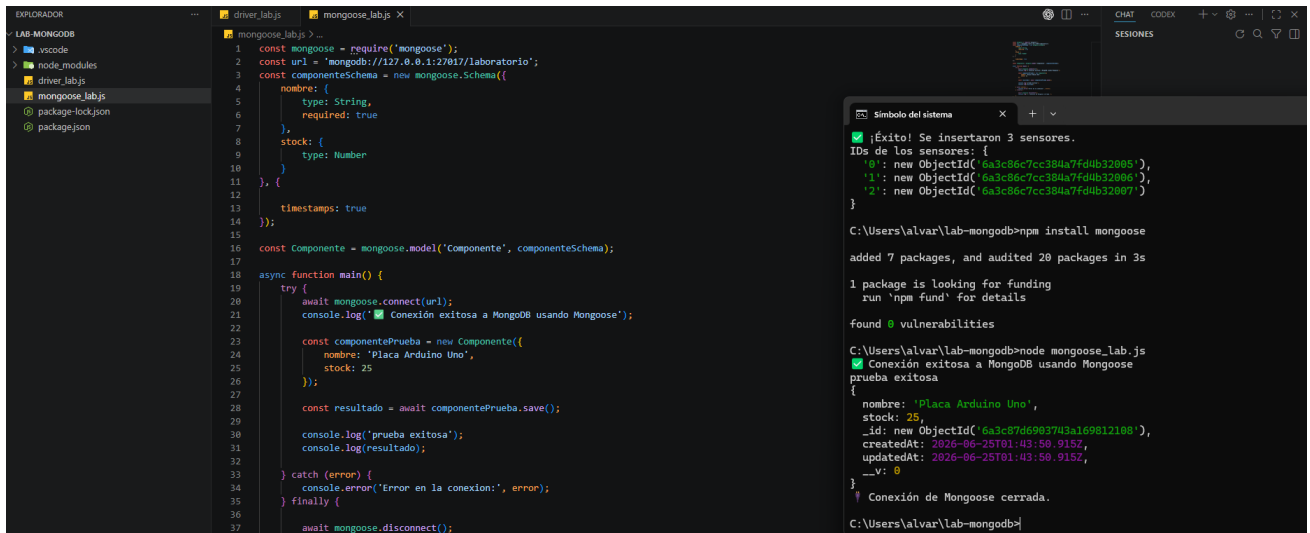
C:\Users\alvar\lab-mongodb>code .
C:\Users\alvar\lab-mongodb>node driver_lab.js
[Exito] Se insertaron 3 sensores.
IDs de los sensores: [
  '0': new ObjectId('6a3c86c7cc384a7fd4b32085'),
  '1': new ObjectId('6a3c86c7cc384a7fd4b32086'),
  '2': new ObjectId('6a3c86c7cc384a7fd4b32087')
]
```

Parte 2: Implementación con Mongoose (ODM)

Mongoose ayuda a mantener el orden y la coherencia en aplicaciones de mediana o gran escala.

Instrucciones:

1. Instalen el paquete: `npm install mongoose`.
2. Definan un **Esquema (Schema)** para los componentes que incluya:
 - **nombre**: Tipo **String** y obligatorio (**required: true**).
 - **stock**: Tipo **Number**.
3. Activen la opción **timestamps: true** para generar automáticamente los campos de fecha de creación y actualización.
4. Creen el modelo y realicen una inserción de prueba.



```
1 const mongoose = require('mongoose');
2 const uri = 'mongodb://127.0.0.1:27017/laboratorio';
3 const componenteSchema = new mongoose.Schema({
4   nombre: {
5     type: String,
6     required: true
7   },
8   stock: {
9     type: Number
10  },
11 }, {
12   timestamps: true
13 });
14
15 const Componente = mongoose.model('Componente', componenteSchema);
16
17 async function main() {
18   try {
19     await mongoose.connect(uri);
20     console.log('Conexión exitosa a MongoDB usando Mongoose');
21
22     const componentePrueba = new Componente({
23       nombre: 'Placa Arduino Uno',
24       stock: 25
25     });
26
27     const resultado = await componentePrueba.save();
28
29     console.log('prueba exitosa');
30     console.log(resultado);
31
32   } catch (error) {
33     console.error('Error en la conexión:', error);
34   } finally {
35     await mongoose.disconnect();
36   }
37 }
```

```
¡Éxito! Se insertaron 3 sensores.
IDs de los sensores: {
  '0': new ObjectId('6a3c86c7cc384a7fd4b32805'),
  '1': new ObjectId('6a3c86c7cc384a7fd4b32806'),
  '2': new ObjectId('6a3c86c7cc384a7fd4b32807')
}

C:\Users\alvar\lab-mongodb>npm install mongoose
added 7 packages, and audited 20 packages in 3s
1 package is looking for funding
run 'npm fund' for details
found 0 vulnerabilities

C:\Users\alvar\lab-mongodb>node mongoose_lab.js
Conexión exitosa a MongoDB usando Mongoose
prueba exitosa
{
  nombre: 'Placa Arduino Uno',
  stock: 25,
  _id: new ObjectId('6a3c87d66901743a169812100'),
  createdAt: 2026-06-25T01:43:50.915Z,
  updatedAt: 2026-06-25T01:43:50.915Z,
  __v: 0
}
Conexión de Mongoose cerrada.

C:\Users\alvar\lab-mongodb>
```

Parte 3: Reflexión Final

Basándose en lo aprendido, respondan:

1. ¿Cuál es la función del driver como "traductor" entre la aplicación y MongoDB?
El driver actúa como intermediario entre la aplicación y MongoDB. Convierte las instrucciones escritas en Node.js (como `find` o `insertMany`) en comandos que MongoDB puede entender y ejecutar, además de gestionar la conexión con la base de datos.
2. ¿A qué formato convierte el driver las estructuras de datos nativas (como objetos de JavaScript) para enviarlas al servidor?

Las convierte a BSON (Binary JSON). Este formato es más eficiente que JSON porque ocupa menos espacio, se procesa más rápido y permite manejar tipos de datos especiales como `ObjectId` y fechas.

3. Según lo visto, ¿en qué situación de un proyecto de ingeniería elegirían usar el driver nativo en lugar de Mongoose?

Se utiliza el driver nativo cuando se busca el máximo rendimiento y velocidad, por ejemplo en sistemas que reciben grandes volúmenes de datos de sensores o dispositivos IoT. Al no realizar validaciones ni usar esquemas, procesa las operaciones más rápido que Mongoose.